

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Analytical Problem Solving: 40%

QUESTIONS:

Total Marks: 50

Annexure A

Code of Conduct:

I _Pusalapati Chandu(192372119)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *Pchandu*

Annexure A

Analytical Problem Solving

1. RSA Key Generation and Security Analysis

Given:

$p = 17, q = 19$

Compute n and $\phi(n)$

Choose $e = 5$ and calculate the private key d

Encrypt the message $M = 7$

Decrypt the ciphertext and verify correctness

Analyze the impact of choosing a small prime number on RSA security

2. ElGamal Encryption Analysis

Given:

$p = 23, g = 5, \text{ private key } x = 7, \text{ random } k = 3, \text{ message } M = 10$

Compute the public key

Perform encryption to generate ciphertext pair $(C1, C2)$

Decrypt the ciphertext to retrieve the original message

Analyze how randomness (k) improves security in ElGamal

3. RSA Digital Signature Verification

Given:

$p = 11, q = 17, e = 7, \text{ message } M = 8$

Compute n and $\phi(n)$

Find private key d

Generate the digital signature for the message

Verify the signature using the public key

Analyze how digital signatures ensure non-repudiation

4. Diffie-Hellman with Attack Resistance Evaluation

Given:

$p = 29, g = 2$

User A private key = 5

User B private key = 12

Compute public keys for both users

Compute the shared secret key

Analyze what happens if an attacker intercepts and replaces public keys
Suggest a cryptographic solution to ensure authenticity

5. Hybrid Encryption System Analysis

Given:

A system uses RSA for key exchange and AES for data encryption

RSA encryption time = 5 ms per key exchange

AES encryption time = 0.5 ms per data block

Total data blocks = 500

Calculate total time taken for encryption

Compare with using only RSA for encrypting all data

Analyze why hybrid encryption is more efficient

Justify its use in modern secure communication systems

Question 1 — RSA Key Generation and Security Analysis

Given: $p = 17$, $q = 19$, $e = 5$, Message $M = 7$

Step 1: Compute n and $\phi(n)$

Formula	$n = p \times q$
Substitute	$n = 17 \times 19 = 323$
Formula	$\phi(n) = (p - 1)(q - 1)$
Substitute	$\phi(n) = 16 \times 18 = 288$

Step 2: Choose e and calculate private key d

$e = 5$ is valid because $\gcd(5, 288) = 1$ (5 shares no common factor with 288).

d is the modular inverse of e modulo $\phi(n)$, i.e. d such that $(e \times d) \bmod \phi(n) = 1$. Using the Extended Euclidean Algorithm on 288 and 5:

$$288 = 57(5) + 3 \quad 5 = 1(3) + 2 \quad 3 = 1(2) + 1 \quad 2 = 2(1) + 0$$

Back-substituting gives $1 = 2(288) - 115(5)$, so $-115 \equiv 173 \pmod{288}$.

Result	$d = 173$
Check	$(5 \times 173) \bmod 288 = 865 \bmod 288 = 1 \quad \checkmark$

Public Key = $(e, n) = (5, 323)$ Private Key = $(d, n) = (173, 323)$

Step 3: Encrypt the message $M = 7$

Formula	$C = M^e \bmod n$
Substitute	$C = 7^5 \bmod 323$
Working	$7^2 = 49$; $7^4 = 49^2 \bmod 323 = 2401 \bmod 323 = 140$
Working	$7^5 = 140 \times 7 \bmod 323 = 980 \bmod 323 = 11$
Ciphertext	$C = 11$

Step 4: Decrypt the ciphertext and verify correctness

Decryption uses $M = C^d \bmod n = 11^{173} \bmod 323$. This is computed efficiently with the Chinese Remainder Theorem (CRT) using $p = 17$ and $q = 19$:

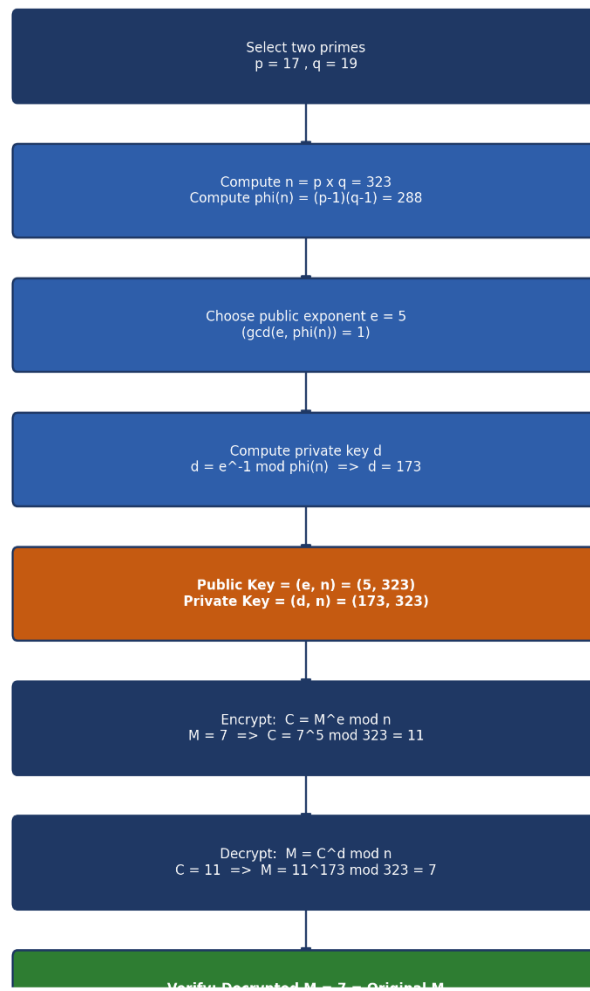
$$\begin{aligned} d \bmod (p-1) &= 173 \bmod 16 = 13 & d \bmod (q-1) &= 173 \bmod 18 = 11 & M_p &= \\ 11^{13} \bmod 17 &= 7 & M_q &= 11^{11} \bmod 19 = 7 & \text{Combine via CRT} &\Rightarrow M = 7 \\ & & & & \text{(consistent mod 17 and mod 19)} \end{aligned}$$

Result

Decrypted message $M = 7$ = Original message $M = 7$

The RSA encryption/decryption cycle is verified as correct.

RSA Key Generation, Encryption & Decryption — Flowchart



Step 5: Impact of choosing small primes on RSA security

Analysis / Interpretation

With $p = 17$ and $q = 19$, $n = 323$ can be factored almost instantly by trial division, which immediately exposes $\phi(n)$ and hence the private key d — completely breaking the scheme. RSA's security rests entirely on the practical difficulty of factoring n ; that difficulty only exists when p and q are large (in real deployments, 1024+ bits each, so n is 2048+ bits). Small primes also make the system vulnerable to Fermat's factorization method (fast when p and q are close in value), Pollard's rho/ $p-1$ algorithms, and simple brute-force search. In addition, small n limits the message space and can expose the scheme to low-exponent attacks. This problem uses small numbers purely to keep the arithmetic tractable by hand — a production RSA system must never use primes this small.

Question 2 — ElGamal Encryption Analysis

Given: $p = 23$, $g = 5$, private key $x = 7$, random $k = 3$, message $M = 10$

Step 1: Compute the public key

Formula	$y = g^x \bmod p$
Substitute	$y = 5^7 \bmod 23$
Working	$5^2 = 25 \bmod 23 = 2$; $5^4 = 2^2 = 4$
Working	$5^7 = 5^4 \times 5^2 \times 5^1 = 4 \times 2 \times 5 = 40 \bmod 23 = 17$
Public key	$y = 17$ (Public: $p=23$, $g=5$, $y=17$ Private: $x=7$)

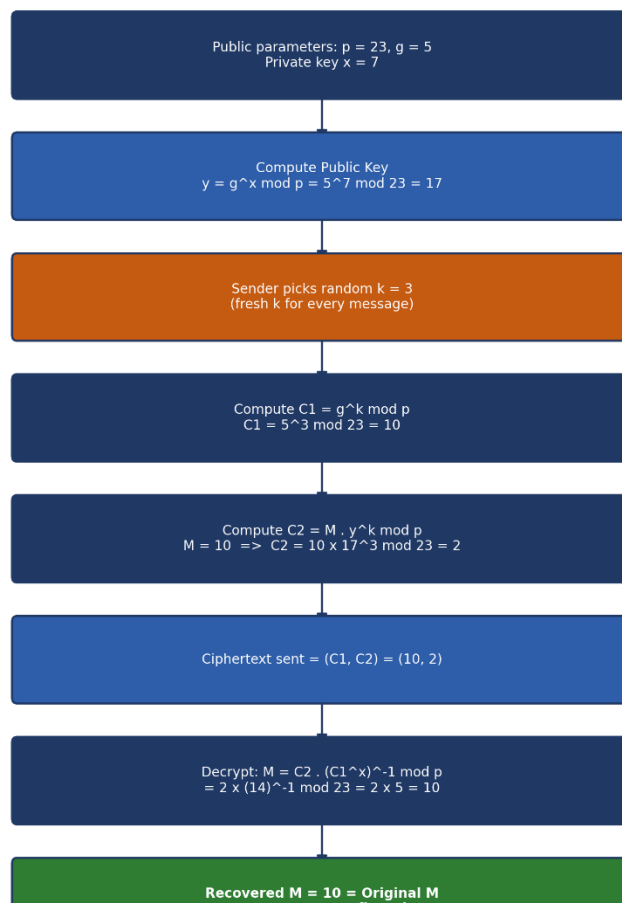
Step 2: Encrypt to generate ciphertext pair (C1, C2)

Formula	$C1 = g^k \bmod p$
Substitute	$C1 = 5^3 \bmod 23 = 125 \bmod 23 = 10$
Formula	$C2 = (M \times y^k) \bmod p$
Working	$y^k = 17^3 \bmod 23$: $17^2 = 289 \bmod 23 = 13$; $17^3 = 13 \times 17 = 221 \bmod 23 = 14$
Substitute	$C2 = (10 \times 14) \bmod 23 = 140 \bmod 23 = 2$
Ciphertext	$(C1, C2) = (10, 2)$

Step 3: Decrypt the ciphertext to retrieve M

Formula	$M = C2 \times (C1^x)^{-1} \bmod p$
Working	$C1^x = 10^7 \bmod 23 = 14$ (by repeated squaring)
Working	Inverse of $14 \bmod 23$ (Extended Euclid) = 5, since $14 \times 5 = 70 \bmod 23 = 1$
Substitute	$M = (2 \times 5) \bmod 23 = 10$
Result Recovered message $M = 10$ = Original message $M = 10$ ElGamal decryption is verified as correct.	

ElGamal Encryption & Decryption — Flowchart



Step 4: How randomness (k) improves security

Analysis / Interpretation

ElGamal is probabilistic rather than deterministic: because a fresh random k is chosen for every encryption, the same plaintext M encrypted twice produces two completely different ciphertext pairs $(C1, C2)$. This defeats pattern-matching and chosen-plaintext attacks that would succeed against a deterministic cipher, and it hides any statistical relationship between repeated plaintexts. However, k must never be reused across two messages under the same key — reusing k lets an attacker who knows one plaintext algebraically recover both k and the private key x , so a cryptographically secure random number generator is essential in any real ElGamal implementation.

Question 3 — RSA Digital Signature Verification

Given: $p = 11$, $q = 17$, $e = 7$, message $M = 8$

Step 1: Compute n and $\phi(n)$

Formula	$n = p \times q = 11 \times 17 = 187$
Formula	$\phi(n) = (p-1)(q-1) = 10 \times 16 = 160$

Step 2: Find the private key d

d is the modular inverse of $e = 7$ modulo $\phi(n) = 160$, found via the Extended Euclidean Algorithm:

$$\begin{aligned} 160 &= 22(7) + 6 & 7 &= 1(6) + 1 & 6 &= 6(1) + 0 & \text{Back-substitute: } 1 &= \\ 23(7) - 160 &=> d &= 23 \end{aligned}$$

Result	$d = 23$
Check	$(7 \times 23) \bmod 160 = 161 \bmod 160 = 1 \quad \checkmark$

Public Key = $(e, n) = (7, 187)$ Private Key = $(d, n) = (23, 187)$

Step 3: Generate the digital signature

In RSA signing, the sender signs with the PRIVATE key: $S = M^d \bmod n$. Using CRT for efficiency:

$$\begin{aligned} d \bmod (p-1) &= 23 \bmod 10 = 3 & d \bmod (q-1) &= 23 \bmod 16 = 7 & S_p &= 8^3 \\ \bmod 11 &= 6 & S_q &= 8^7 \bmod 17 = 15 & \text{Combine via CRT} &=> S = 83 \end{aligned}$$

Signature	$S = 83$
-----------	----------

Step 4: Verify the signature using the public key

The receiver verifies with the PUBLIC key: $M' = S^e \bmod n$.

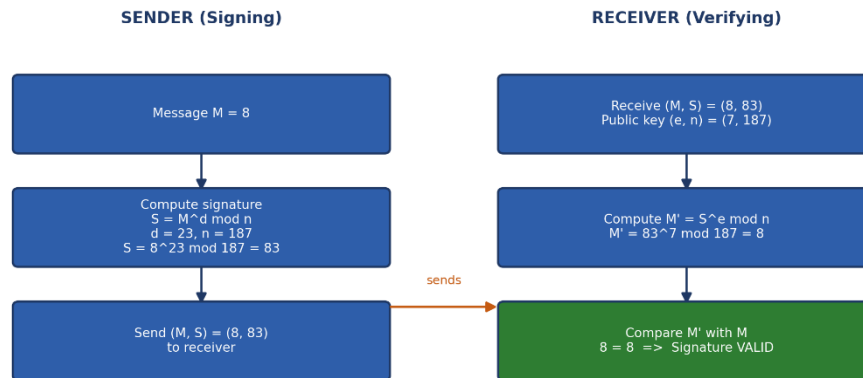
$$\begin{aligned} M' &= 83^7 \bmod 187 & e \bmod (p-1) &= 7 \bmod 10 = 7 & e \bmod (q-1) &= 7 \bmod 16 = 7 \\ M'_p &= 83^7 \bmod 11 = 8 & M'_q &= 83^7 \bmod 17 = 8 & \text{Combine via CRT} &=> M' = 8 \end{aligned}$$

Result

Recovered $M' = 8$ = Original message $M = 8$

Signature is VALID — the message was signed by the holder of the private key and has not been altered.

RSA Digital Signature — Generation & Verification



Step 5: How digital signatures ensure non-repudiation

Analysis / Interpretation

A digital signature is generated using information only the sender possesses — their private key — so a valid signature could only have been produced by that key holder. Because the sender cannot plausibly deny having performed an action that only their private key could have produced, they cannot later repudiate (deny) having sent the message; this property is called non-repudiation. This differs from a simple checksum or MAC, which only proves data integrity between two parties who already share a secret — a MAC does not identify who produced it. RSA signatures additionally provide authentication (proof of the signer's identity) and integrity (any change to M changes S^e and verification fails), and in legal/financial systems (e-contracts, digital certificates, code signing) non-repudiation is what allows a signature to carry evidentiary weight.

Question 4 — Diffie-Hellman with Attack Resistance Evaluation

Given: $p = 29$, $g = 2$, User A private key $a = 5$, User B private key $b = 12$

Step 1: Compute public keys for both users

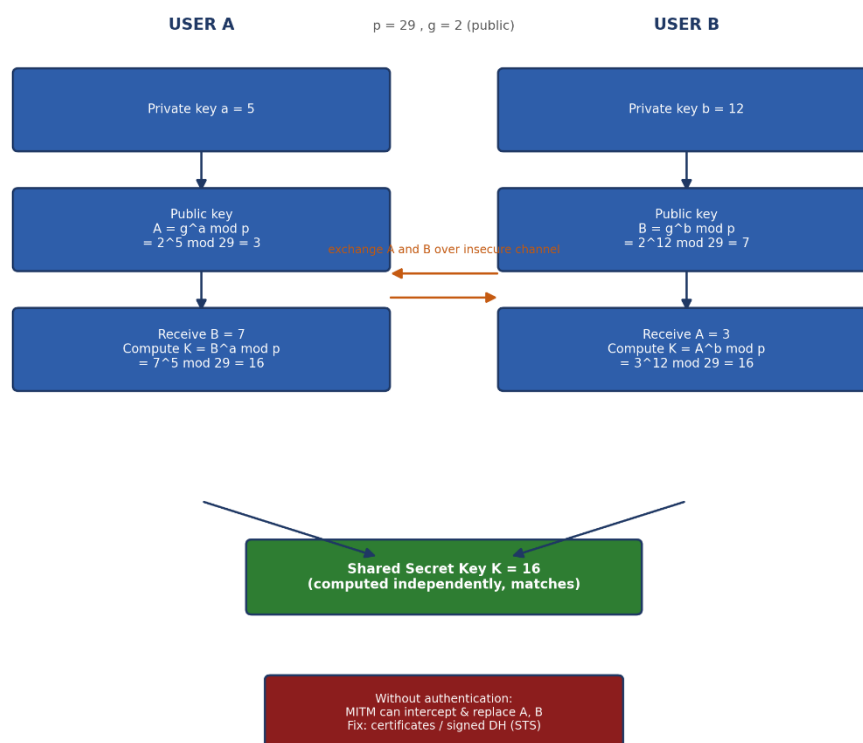
Formula (A)	$A = g^a \bmod p$
Substitute	$A = 2^5 \bmod 29 = 32 \bmod 29 = 3$
Formula (B)	$B = g^b \bmod p$
Working	$2^4 = 16$; $2^8 = 256 \bmod 29 = 24$; $2^{12} = 2^8 \times 2^4 = 24 \times 16 = 384 \bmod 29 = 7$
Substitute	$B = 7$

User A sends $A = 3$ to User B. User B sends $B = 7$ to User A.

Step 2: Compute the shared secret key

A computes	$K = B^a \bmod p = 7^5 \bmod 29 = 16$
B computes	$K = A^b \bmod p = 3^{12} \bmod 29 = 16$
Shared Secret	$K = 16$ (both sides agree ✓)

Diffie-Hellman Key Exchange



Step 3: Impact of an attacker intercepting and replacing public keys

Analysis / Interpretation

Plain Diffie-Hellman guarantees only key agreement, not authentication of the two parties. If an active attacker (Man-in-the-Middle) intercepts A's public value and substitutes their own value going to B, and intercepts B's public value and substitutes their own going to A, then A actually establishes a shared key with the attacker, and B independently establishes a different shared key with the attacker as well — while both A and B believe they are talking directly to each other. The attacker can then transparently decrypt, read, and re-encrypt every message flowing between them. Nothing in the basic DH exchange lets either party verify whose public value they actually received, which is exactly the gap this attack exploits.

Step 4: Cryptographic solution to ensure authenticity

Analysis / Interpretation

Authenticity can be added on top of DH in several standard ways: (1) Authenticated / Station-to-Station (STS) protocol, where each party digitally signs their DH public value (e.g. with RSA or ECDSA) so the receiver can verify it truly came from the claimed sender; (2) Certificate-based DH, where public values are bound to identities using CA-issued digital certificates (the model used in TLS's ephemeral Diffie-Hellman, DHE/ECDHE); (3) A Pre-Shared Key (PSK) or password-authenticated key exchange (PAKE, e.g. SPEKE, SRP) so both sides prove knowledge of a shared secret during the exchange. Modern TLS 1.3 uses certificate-authenticated ephemeral DH precisely to combine DH's forward secrecy with protection against man-in-the-middle attacks.

Question 5 — Hybrid Encryption System Analysis

Given: RSA (key exchange) = 5 ms per exchange, AES (data) = 0.5 ms per block, Total blocks = 500

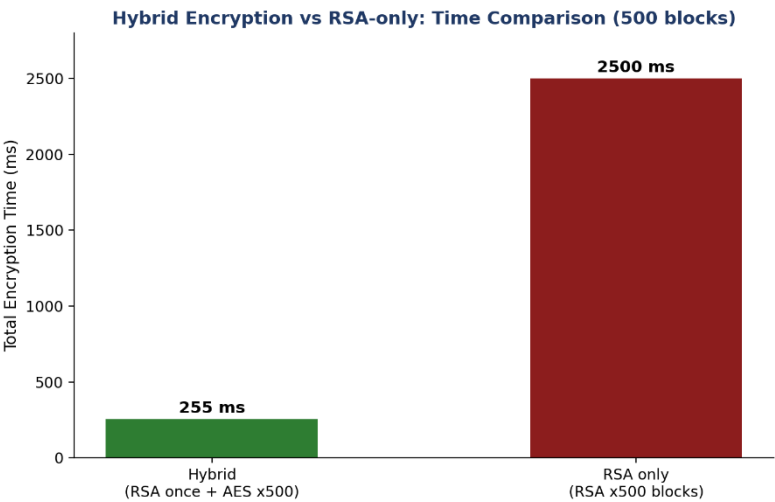
Step 1: Calculate total time taken for hybrid encryption

In hybrid encryption, RSA is used only ONCE — to securely exchange a symmetric AES session key. All bulk data is then encrypted with the much faster AES algorithm.

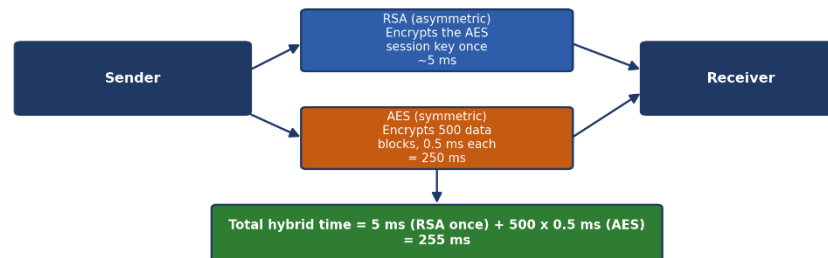
RSA cost	1 key exchange × 5 ms = 5 ms
AES cost	500 blocks × 0.5 ms = 250 ms
Total Hybrid Time	5 ms + 250 ms = 255 ms

Step 2: Compare with using only RSA for all data

RSA-only cost	500 blocks × 5 ms/block = 2500 ms
Time saved by hybrid	2500 - 255 = 2245 ms
Efficiency gain	≈ 89.8% reduction in total encryption time



Hybrid Encryption (RSA + AES) — Architecture



Step 3 & 4: Why hybrid encryption is more efficient, and its role in modern systems

Analysis / Interpretation

RSA (asymmetric encryption) is computationally expensive because it relies on large-integer modular exponentiation, while AES (symmetric encryption) uses simple substitution-permutation rounds that are orders of magnitude faster in hardware and software — the numbers above show RSA costs 10x more per block than AES (5 ms vs 0.5 ms). Using RSA for every one of the 500 blocks would multiply that expensive operation 500 times, whereas hybrid encryption pays the RSA cost exactly once (to protect a short session key) and lets AES — which is both fast and, at 256-bit strength, extremely secure — handle the bulk data. This combination captures the best of both worlds: RSA's public-key property solves the key-distribution problem (no need to pre-share a secret), while AES delivers the throughput real systems need. This is precisely why hybrid encryption is the standard in modern secure communication — TLS/SSL (HTTPS), PGP/GPG email encryption, SSH, and VPN protocols all use an asymmetric algorithm only to establish or wrap a symmetric session key, then switch to a symmetric cipher for the actual data transfer.

Summary of Final Answers

- Q1 (RSA): $n=323$, $\phi(n)=288$, $d=173$, $C=11$, Decrypted $M=7$ — matches original.
Q2 (ElGamal): $y=17$, Ciphertext $(C1,C2)=(10,2)$, Decrypted $M=10$ — matches original.
Q3 (Signature): $n=187$, $d=23$, Signature $S=83$, Verified $M'=8$ — Signature VALID.
Q4 (Diffie-Hellman): $A=3$, $B=7$, Shared Secret $K=16$ — matches on both sides.
Q5 (Hybrid): Hybrid total = 255 ms vs RSA-only = 2500 ms — hybrid is ~89.8% faster.

